Guia: eco da Alexa no PC (Omarchy)

Este guia conta o que aconteceu em 12/10/2026: o som que o PC mandava pro Echo Dot (Alexa) voltava pelo microfone do notebook. Aqui está o porquê, como as caixas inteligentes resolvem isso, o que foi feito no PC, como foi testado e como desfazer.

1. O problema e por que aconteceu

O PC (notebook com Arch Linux/Omarchy) manda o som pro Echo Dot por **Bluetooth**. O Echo toca esse som no quarto, e o **microfone do notebook** escuta tudo de novo. Resultado: quem estava numa chamada ou gravação ouvia a música/voz de volta, como um eco.

Um PC comum não sabe sozinho que precisa "descontar" o que ele mesmo está tocando. Pra isso existe o **cancelamento de eco acústico (AEC)**, que não estava ligado.

Um detalhe importante que as medições mostraram: entre o som sair do PC pelo Bluetooth e chegar ao microfone passam de **140 a 490 milissegundos**, e esse tempo **varia**. Isso é culpa do Bluetooth (A2DP) e do processamento do próprio Echo Dot. Quanto maior e mais instável esse atraso, mais difícil cancelar o eco.

2. Por que a correção do anti-rev não era ideal

Antes, outro bot (o anti-rev) tinha montado um esquema pra controlar o volume do Echo pelo PC:

- criou uma saída virtual chamada "Echo" (echo_softvol) e deixou como saída padrão;
- ligou essa saída ao Bluetooth com um **loopback de 50 ms** (latency_msec=50).

Problemas: esse loopback **somava 50 ms de atraso** em tudo o que se ouvia, e **não havia cancelamento de eco nenhum**. O microfone continuava sendo o microfone "cru", então o eco continuava.

Na primeira tentativa de correção, liguei o cancelador de eco pelo comando pactl (módulo module-echo-cancel) com opções como extended_filter e delay_agnostic. Ele **quase não funcionou: só 0,5 a 2,5 dB de redução**. Motivo: essas opções **não existem** na versão instalada (PipeWire 1.6.8 com WebRTC 1.3, que usa o AEC3) e foram ignoradas, e esse jeito de carregar o módulo não permite ajustar o alinhamento da referência, que era justamente o que faltava.

3. Como a Alexa cancela o eco

A Alexa precisa ouvir "Alexa" mesmo tocando música alta. Ela usa várias técnicas juntas:

- **Sinal de referência:** ela sabe exatamente o que está tocando, porque é ela quem toca. Essa cópia é a "referência".
- **Filtro adaptativo (AEC):** um algoritmo aprende como aquele som sai do alto-falante, bate nas paredes e chega ao microfone. Ele prevê o eco e **subtrai** do que o microfone captou. Para isso a referência precisa estar **alinhada no tempo** com o eco real.
- **Supressão de eco residual:** um segundo filtro, depois do AEC, que apaga o restinho que sobrou.
- **Beamforming:** a Alexa tem 6 ou 7 microfones e combina todos para "apontar" a escuta para quem fala, ignorando outras direções.
- **Barge-in:** poder interromper a Alexa falando por cima dela enquanto ela toca. Só funciona graças ao AEC.
- **Full-duplex:** falar e ouvir ao mesmo tempo, como numa ligação normal, sem "um de cada vez".

4. O que foi feito no PC, passo a passo

Tudo foi feito no usuário (sem sudo), sem reiniciar o sistema de áudio.

Passo 1: tirar o loopback de 50 ms

A saída virtual + loopback do anti-rev foram removidos. Agora o som vai **direto** do cancelador pro Bluetooth, sem atraso fixo.

Passo 2: cancelador de eco nativo do PipeWire

Em vez do módulo pelo pactl, o script inicia um **processo próprio do PipeWire** com o cancelador WebRTC. Opções usadas:

- **referência atrasada em 100 ms** (buffer.play_delay = 4800/48000). Isso atrasa **só a cópia** que o cancelador usa para comparar, **não o som que você ouve**;
- supressão de ruído ligada e filtro passa-alta (corta graves/zumbido) ligado;
- controle automático de ganho desligado (evita o volume do mic ficar "respirando").

O script que monta tudo fica em:

```
~/.local/bin/echo-softvol-up
```

Ele gera este arquivo de configuração toda vez que roda:

```
~/.config/echo-softvol/aec.conf
```

Conteúdo (resumido; MIC = microfone do notebook, BT_SINK = saída Bluetooth do Echo):

```
context.modules = [
 { name = libpipewire-module-echo-cancel args = {
   library.name = aec/libspa-aec-webrtc
   monitor.mode = false
   buffer.play_delay = 4800/48000
   aec.args = { webrtc.gain_control = false
                webrtc.high_pass_filter = true
                webrtc.noise_suppression = true }
   capture.props  = { node.name = echo-cancel-capture  target.object = MIC }
   source.props   = { node.name = echo_mic      node.description = Mic-sem-
eco }
```

```
    sink.props     = { node.name = echo_softvol  node.description = Echo }
    playback.props = { node.name = echo-cancel-playback target.object =
BT_SINK }
 } }
]
```

E inicia o processo assim:

```
pipewire -c ~/.config/echo-softvol/aec.conf
```

Passo 3: serviço automático

O serviço echo-softvol-watcher (criado pelo anti-rev e mantido) roda o script sozinho quando o Echo conecta e desmonta tudo quando desconecta. Os nomes echo_softvol e echo_mic foram mantidos para ele continuar funcionando. Para ver se está ativo:

```
systemctl --user status echo-softvol-watcher
```

Passo 4: aparelhos padrão

- **Saída padrão:** "Echo" (echo_softvol). O controle de volume pelo PC continua funcionando.
- **Entrada padrão:** "Mic-sem-eco" (echo_mic). É o microfone já limpo.

```
pactl get-default-sink     # deve mostrar echo_softvol
pactl get-default-source   # deve mostrar echo_mic
```

5. Como testar

Teste simples (qualquer pessoa)

1. No app de chamada ou gravação, escolha o microfone "Mic-sem-eco".
2. Toque música alta no Echo e fale normalmente.
3. Grave ou peça para alguém na chamada dizer se ouve a música de volta.
4. Desconecte e reconecte o Echo e confira se tudo volta sozinho.

Nos primeiros segundos depois de conectar pode passar um eco leve: o cancelador está "aprendendo" o quarto.

Como foi medido

Em vez de "achismo", foi feito um teste objetivo com scripts em ~/aec-test/:

- o PC toca um **chirp** (um bipe que sobe de grave para agudo) e depois um ruído parecido com música, em volume moderado;
- ao mesmo tempo grava três coisas: o microfone cru, o microfone limpo (echo_mic) e o que sai pro Bluetooth;
- com **correlação cruzada** (uma conta que procura onde dois sinais "se encaixam") descobre-se quanto tempo o som leva do Bluetooth ao microfone;
- comparando o volume do eco no mic cru e no mic limpo, sai a **redução em dB** (cada 10 dB a menos ≈ metade do volume percebido; 20–30 dB é uma redução grande).

```
cd ~/aec-test
SRC=echo_mic SNK=echo_softvol ./t2.sh
```

Resultados por rodada

Configuração	Bluetooth → mic	Redução do eco (bipe / música)	Eco
1ª tentativa: módulo pelo pactl	180–490 ms	0,4–1,2 / 2,4–2,5 dB	presente
PipeWire nativo, sem alinhamento	~180 ms	5–15 / 9–14 dB	presente → leve
Nativo, referência +50 ms	~165 ms	22 / 18,5 dB	ausente
Nativo, referência +100 ms (escolhida)	**140–165 ms**	**24–31 / 12–23 dB**	**ausente**
Nativo, referência +200 ms	~150 ms	8 / 12 dB	leve
Nativo, referência +400 ms	135–190 ms	3,5 / 5 dB	presente

Com a configuração final, o eco ficou **abaixo do ruído de fundo do microfone**, ou seja, inaudível nos testes. O atraso de quem ouve não aumentou com o ajuste de 100 ms; o que diminuiu foi o atraso total, porque os 50 ms do loopback saíram. Os números variam de uma medição para outra porque o atraso do Bluetooth oscila e o cancelador leva alguns segundos para se ajustar.

6. Limites e como desfazer

Limite do Bluetooth

Os **140–490 ms** de atraso vêm do Bluetooth A2DP e do Echo Dot. Nenhum programa no PC consegue tirar isso. Um cabo eliminaria quase tudo, mas o Echo Dot só tem saída P2 (Line Out), não entrada, então provavelmente não dá para ligar o PC nele por cabo.

Limite de um microfone só

O notebook tem um microfone, sem beamforming como a Alexa. Com som muito alto ou alguém falando junto, o cancelamento piora um pouco.

Como desfazer

Backups guardados na pasta pessoal:

- ~/echo-softvol-up.bak-anti-rev: versão original do anti-rev (com o loopback de 50 ms)
- ~/echo-softvol-up.bak-round0: primeira tentativa (módulo pelo pactl)
- ~/echo-softvol-up.bak-round1: versão antes da troca para o PipeWire nativo

Para voltar a uma versão (exemplo: a original do anti-rev):

```
cp ~/echo-softvol-up.bak-anti-rev ~/.local/bin/echo-softvol-up
~/.local/bin/echo-softvol-up
```

Para testar outro alinhamento sem editar nada (exemplo: 50 ms):

```
PLAY_DELAY=2400/48000 ~/.local/bin/echo-softvol-up
```

Para ficar permanente, troque o valor de PLAY_DELAY dentro do script.